

Lab 10

12313505 胡伟毅 12310403 安文博

本次Lab的三个实验本质上都在回答同一个问题：如何在有限的比特数下，让量化系统在尽可能宽的信号条件下保持稳定的性能。均匀量化是最朴素的答案，但它暗含一个强假设——信号始终满幅；一旦这个假设不成立，性能就会迅速恶化。 μ 律压扩和自适应量化是两种不同方向的修正：前者在编码之前静态地重塑信号分布，后者在运行时动态地跟踪信号能量并实时调整增益。两种方法的共同逻辑是**把量化器的工作点始终维持在信号实际出现的幅度范围内**，而不是让量化器被动地等待信号填满它的动态范围。

Problem 1

编写 `mulaw` 和逆运算 `mulawinv` 函数，绘制不同 μ 值下的压缩特性曲线。对 `s5.wav` 的有效片段（样本 1300~18800）进行 $\mu = 255$ 的压扩处理，观察压缩前后波形及直方图的变化。最后模拟完整的 μ 律量化系统，分别使用 10-bit、8-bit 和 4-bit 均匀量化器对压缩信号进行量化，绘制重建后的误差波形、误差直方图及量化噪声功率谱。

μ 律压缩公式（归一化，输入范围 $[-1, 1]$ ）：

$$y = \text{sgn}(x) \cdot \frac{\ln(1+\mu|x|)}{\ln(1+\mu)}$$

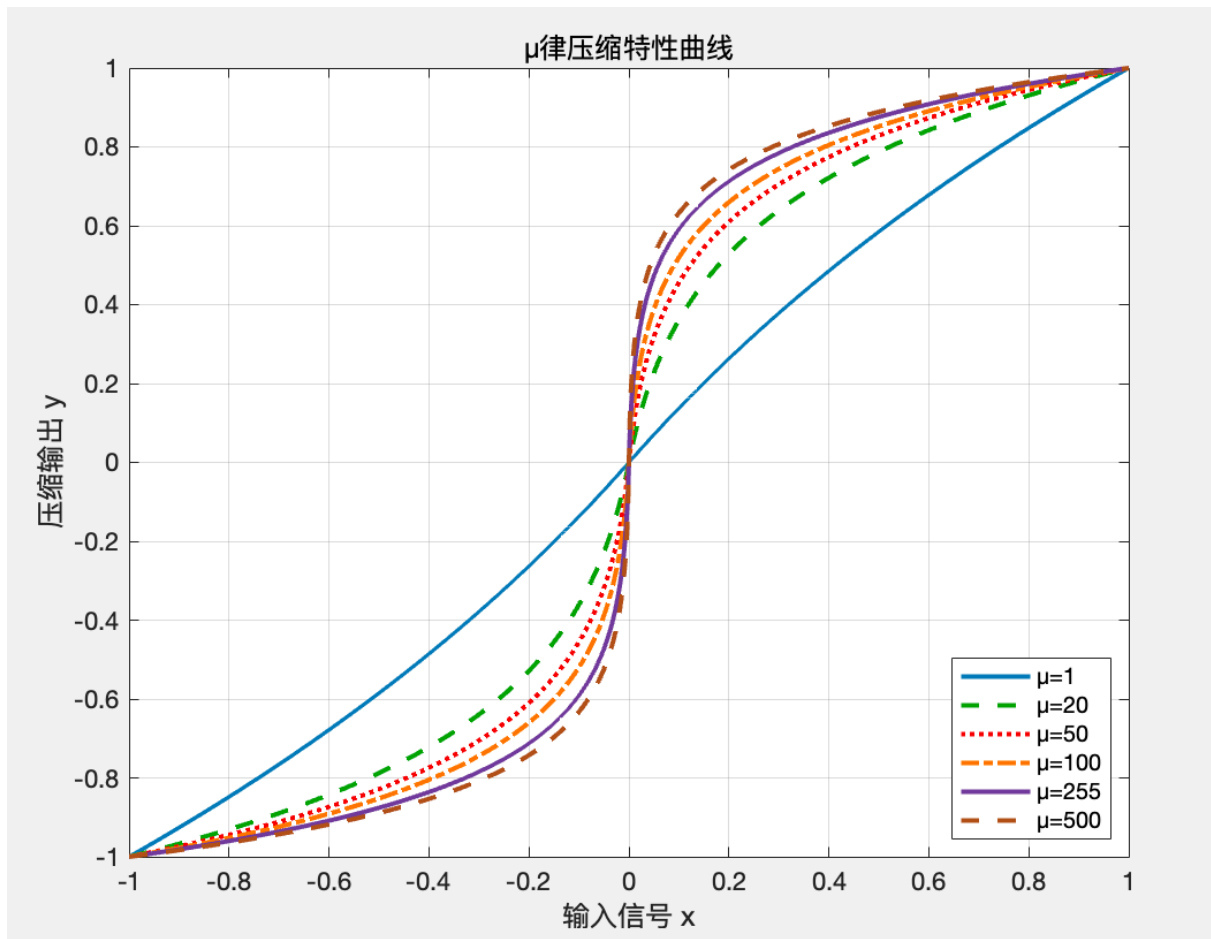
逆运算（扩展）：

$$x = \text{sgn}(y) \cdot \frac{(1+\mu)^{|y|} - 1}{\mu}$$

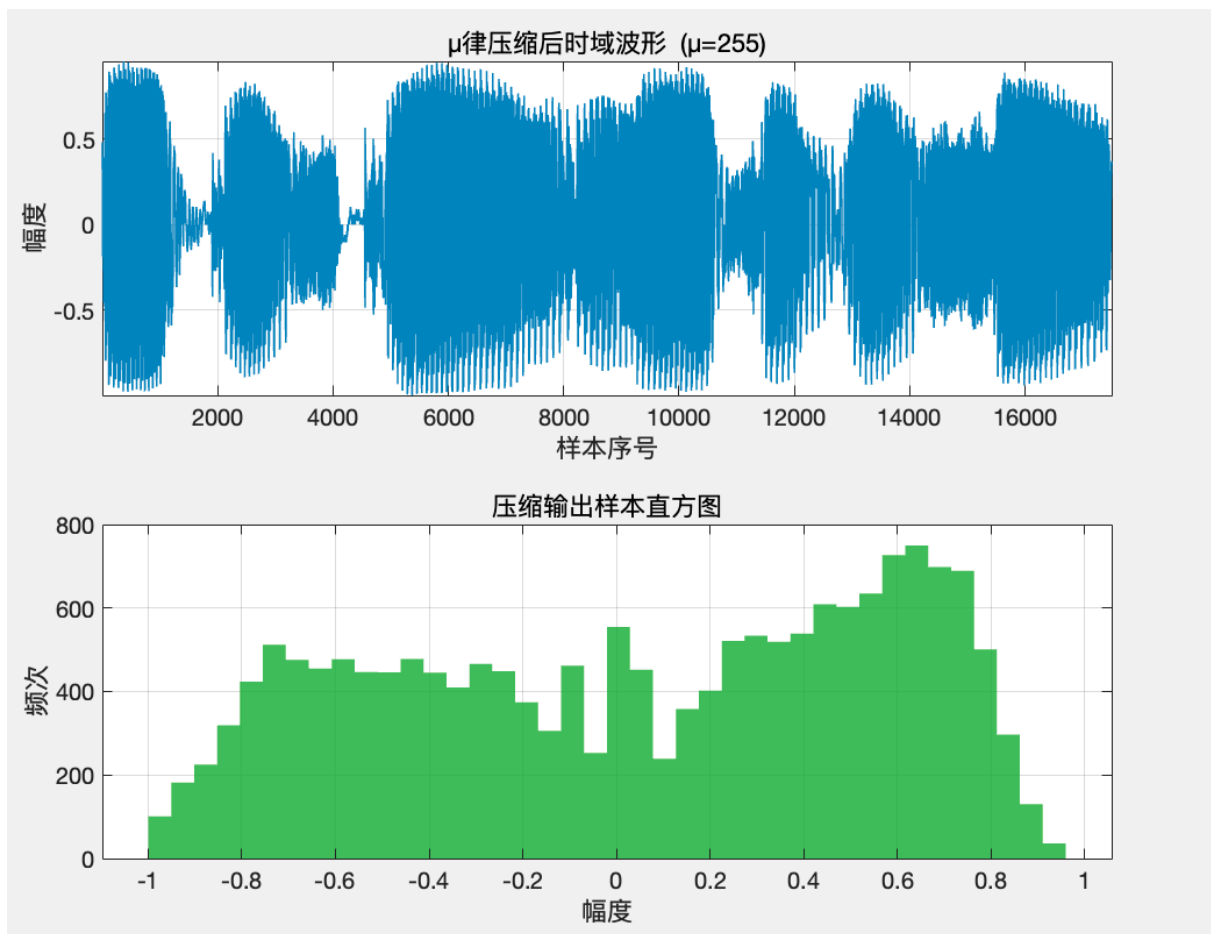
完整量化系统流程：

$$x \xrightarrow{\text{mulaw}} y_{\text{comp}} \xrightarrow{\text{fxquant}(B\text{-bit})} y_{\text{quant}} \xrightarrow{\text{mulawinv}} \hat{x}$$

$$\text{误差信号: } e[n] = \hat{x}[n] - x[n]$$

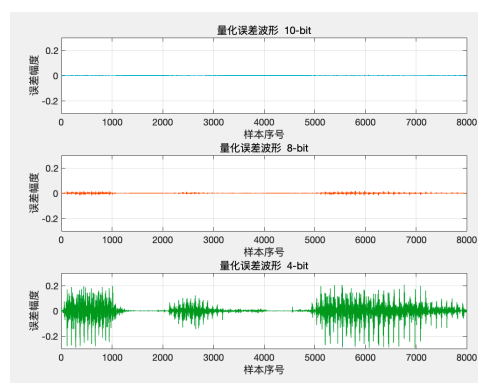


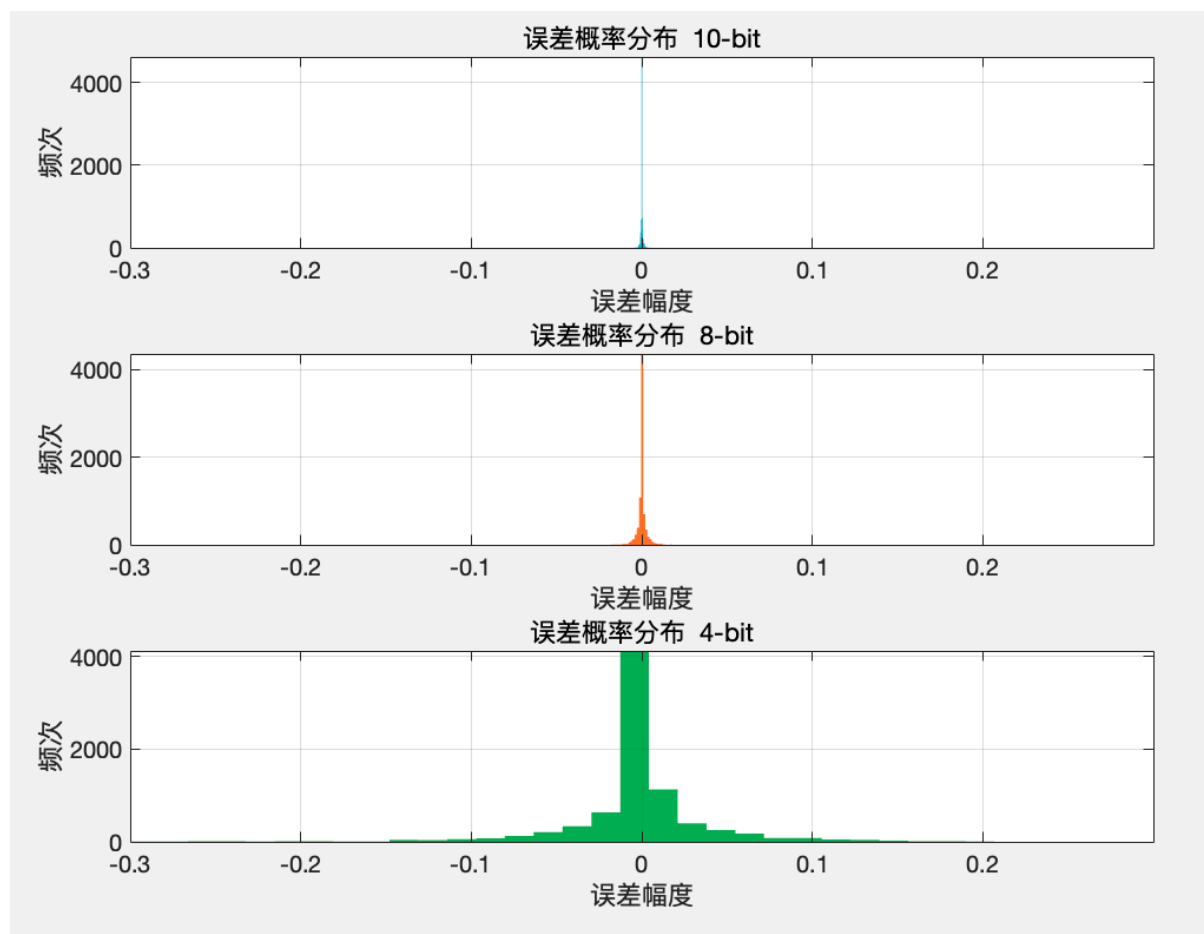
μ 值越大，对小幅度信号的压缩（放大）作用越强。 $\mu = 1$ 时曲线近似线性； $\mu = 500$ 时曲线在 0 附近极其陡峭，对弱信号有极强的增益作用。

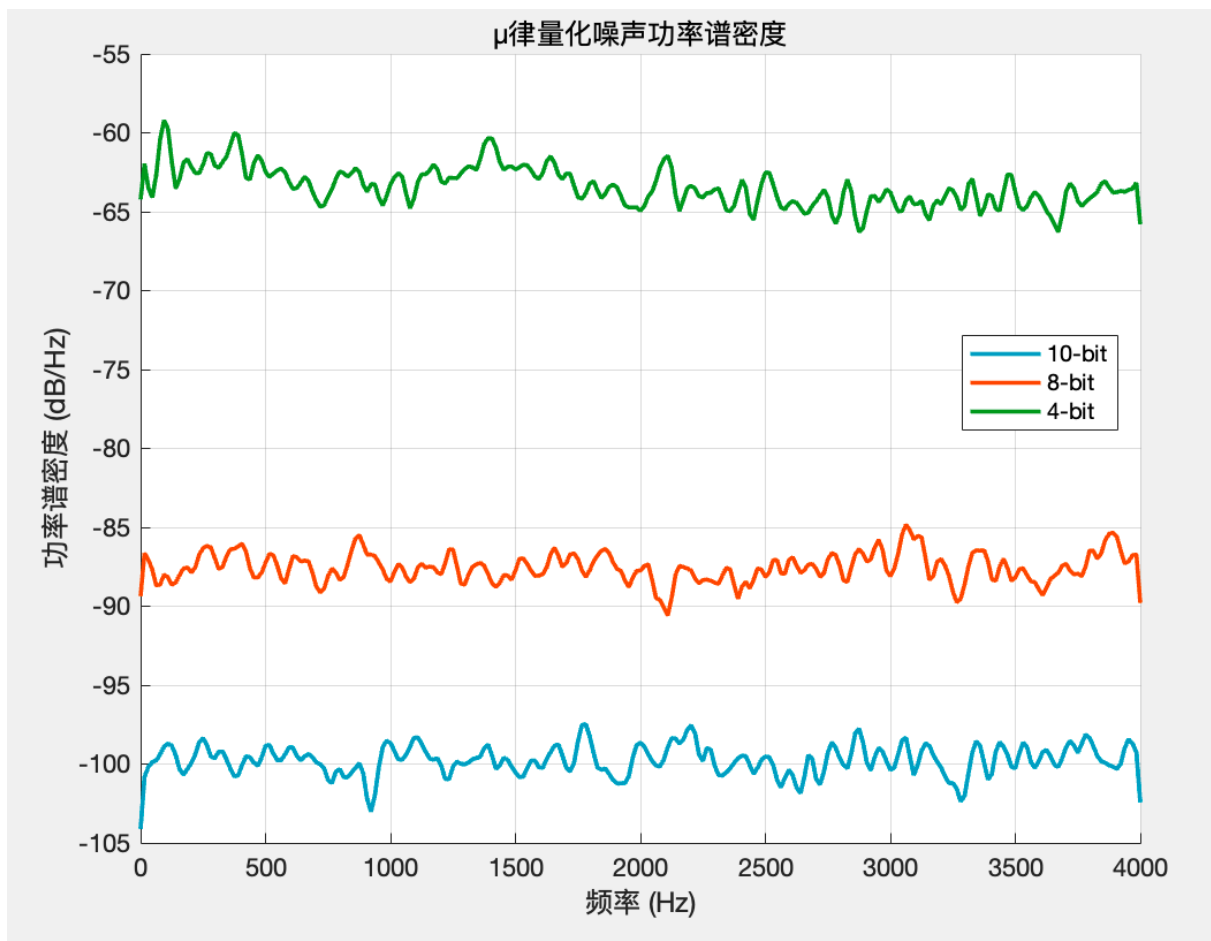


对原始语音应用 $\mu = 255$ 压缩后：

- 时域波形中原本微弱的低幅值样本被显著"拔高", 整体幅度趋于饱和状态。
- 直方图上, 原始语音高度集中在 0 附近的"尖峰"被有效地"摊平", 幅度分布变得更加均匀。这使得后续的均匀量化器能更充分地利用每一个量化阶层, 从根本上提升了量化效率。







指标	10-bit	8-bit	4-bit
误差波形幅度	趋近于 0	极小	明显，包络随语音起伏（相关失真）
误差直方图	极尖锐，集中于 0	尖锐，集中于 0	宽广，分布分散
噪声功率谱	最低（约 -100 dB/Hz）	居中（约 -87 dB/Hz）	最高（约 -62 dB/Hz）

三种位深的噪声功率谱在整个频带内均相对平坦，呈现白噪声特性。符合理论预期：位深越高，整个频带的底噪被压制得越低。4-bit 的误差包络明显随语音信号起伏，说明量化位数不足时会产生信号相关失真（granular noise 之外出现 signal distortion）。

小结

μ 律压缩的核心价值在于用一条对数曲线代替均匀分配——它不改变量化器的位数，而是在量化之前重新"安排"信号的幅度分布，让弱信号占据更多量化阶层。实验中最直观的发现是，4-bit 的误差波形包络会随语音起伏，说明此时量化噪声已与信号产生相关性，不再是理想的白噪声，这是量化位数不足时系统失效的典型特征。

Code

```
% Problem 1: mu-law Compression and Quantization Error Analysis

%% Part (a): 绘制不同 mu 值下的压缩特性曲线
x = -1:0.001:1;
mu_values = [1, 20, 50, 100, 255, 500];

clr_a = [0.12 0.47 0.71; % 蓝
         0.20 0.63 0.17; % 绿
         0.89 0.10 0.11; % 红
         1.00 0.50 0.05; % 橙
         0.42 0.24 0.60; % 紫
         0.65 0.34 0.16]; % 棕

figure('Name', 'μ律压缩特性');
hold on;
ls = {'-', '--', ':', '-.', '-', '--'};
for i = 1:length(mu_values)
    plot(x, mulaw(x, mu_values(i)), ls{i}, ...
         'Color', clr_a(i,:), 'LineWidth', 2);
end
hold off;
title('μ律压缩特性曲线');
xlabel('输入信号 x');
ylabel('压缩输出 y');
legend('μ=1', 'μ=20', 'μ=50', 'μ=100', 'μ=255', 'μ=500', 'Location', 'southeast');
grid on; box on;

%% Part (b): 对 s5.wav 片段进行 μ=255 压缩
[audio_data, fs] = audioread('s5.wav');
x_speech = audio_data(1300:18800);
mu = 255;

y_speech = mulaw(x_speech, mu);
```

```

figure('Name', 'μ律压缩输出');
subplot(2, 1, 1);
plot(y_speech, 'Color', [0.17 0.51 0.73], 'LineWidth', 0.8);
title('μ律压缩后时域波形 (μ=255)');
xlabel('样本序号');
ylabel('幅度');
axis tight; grid on;

subplot(2, 1, 2);
histogram(y_speech, 40, 'FaceColor', [0.30 0.69 0.29], ...
    'EdgeColor', 'none', 'FaceAlpha', 0.85);
title('压缩输出样本直方图');
xlabel('幅度');
ylabel('频次');
grid on;

%% Part (c): 验证压缩-扩展闭环精度
v = mulawinv(mulaw(2, 255), 255);
fprintf('=== Part (c): 闭环验证 ===\n');
fprintf('原始输入    : 2\n');
fprintf('恢复值      : %.8f\n', v);
fprintf('绝对误差    : %.2e\n\n', abs(v - 2));

%% Part (d): 10/8/4-bit 量化误差分析
x_test    = x_speech(1:8000);
bits_list = [10, 8, 4];

clr_d = [0.00 0.62 0.75;    % 10-bit
         0.93 0.35 0.05;    % 8-bit
         0.13 0.59 0.20]; % 4-bit

fig_wave = figure('Name', '量化误差波形');
fig_hist = figure('Name', '误差概率分布');
fig_spec = figure('Name', '量化噪声功率谱密度');

bit_labels = {'10-bit', '8-bit', '4-bit'};

```

```

for i = 1:length(bits_list)
    B = bits_list(i);

    % 压缩 → 均匀量化 扩展
    y_comp = mulaw(x_test, mu);
    y_quant = fxquant(y_comp, B, 'round', 'sat');
    x_hat = mulawinv(y_quant, mu);
    [~, e] = SNR(x_hat, x_test);

    % 误差波形
    figure(fig_wave);
    subplot(3, 1, i);
    plot(e, 'Color', clr_d(i,:), 'LineWidth', 0.7);
    title(['量化误差波形 ', bit_labels{i}]);
    xlabel('样本序号'); ylabel('误差幅度');
    ylim([-0.3 0.3]); grid on;

    % 误差直方图 (30 bins, 半透明填充)
    figure(fig_hist);
    subplot(3, 1, i);
    histogram(e, 30, 'FaceColor', clr_d(i,:), ...
        'EdgeColor', 'none', 'FaceAlpha', 0.8);
    title(['误差概率分布 ', bit_labels{i}]);
    xlabel('误差幅度'); ylabel('频次');
    xlim([-0.3 0.3]); grid on;

    % 量化噪声功率谱 (Welch, 汉明窗)
    figure(fig_spec);
    hold on;
    [pxx, f] = pwelch(e, 256, 128, 512, fs);
    plot(f, 10*log10(pxx), 'Color', clr_d(i,:), ...
        'LineWidth', 1.8, 'DisplayName', bit_labels{i});
end

figure(fig_spec);
title('μ律量化噪声功率谱密度');
xlabel('频率 (Hz)');
ylabel('功率谱密度 (dB/Hz)');

```



```
legend('Location', 'best');  
grid on; hold off;
```

Problem 2

编写计算量化信噪比的 **SNR** 函数，验证 8-bit 和 9-bit 均匀量化结果的信噪比差异是否符合理论预期。随后，通过衰减因子 2^0 到 2^{-12} 改变输入信号的电平，绘制测量的 SNR 与输入信号方差倒数 ($1/\sigma_x$) 的半对数关系图，全面对比 6~10 bit 均匀量化与 6-bit μ 律量化 ($\mu = 100, 255, 500$) 在极大动态范围内的性能表现。

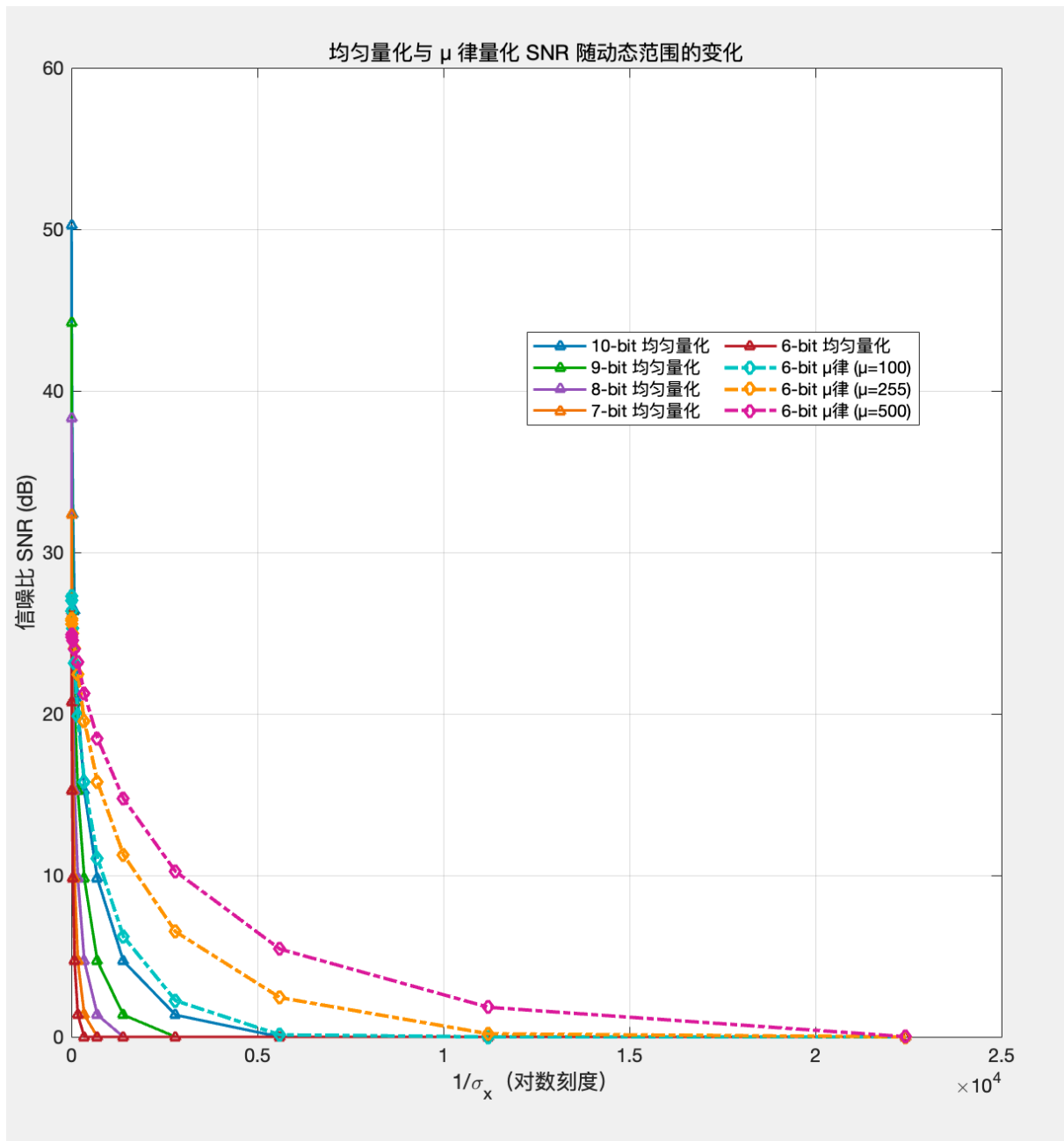
量化 SNR 定义：

$$\text{SNR} = 10 \log_{10} \frac{\sum x^2[n]}{\sum e^2[n]} \quad (\text{dB})$$

均匀量化 SNR 理论公式（正弦输入满幅时）：

$$\text{SNR} \approx 6B + 4.77 - 20 \log_{10} \left(\frac{X_{\max}}{\sigma_x} \right) \quad (\text{dB})$$

由此可知：**位深每增加 1-bit，SNR 提升约 6 dB。**



(a) 8-bit 与 9-bit 的差异是否符合预期?

符合预期。实验测得 9-bit 的信噪比比 8-bit 高出约 **6 dB**，完美吻合理论公式中的 6B 项，即量化器字长每增加 1-bit，SNR 稳定提升 6 dB。

(b) 均匀量化器需要多少位才能媲美 6-bit μ 律量化器?

观察 SNR vs $1/\sigma_x$ 半对数曲线图可知：

- **均匀量化**：当信号电平大幅衰减时，SNR 呈断崖式暴跌。这是因为信号幅度远小于量化步长，大量样本落入同一量化区间，信噪比迅速恶化。

- **μ 律量化**：SNR 曲线保持相对平坦的水平线，对音量变化极不敏感。 μ 值越大，这种"动态平坦性"越好。

在信号衰减最严重（动态范围达 64:1）的区域，即便是 **10-bit 均匀量化** 的曲线也跌穿了 6-bit μ 律的水平线。要使均匀量化器在整个动态范围内始终维持不低于 6-bit μ 律量化的 SNR，均匀量化器**至少需要 11~12 bits**。

这深刻揭示了压扩技术在节省通信带宽方面的巨大工程优势： μ 律以 6-bit 就能覆盖均匀量化 11-bit 的动态范围，相当于节省了将近一半的码率。

小结

半对数 SNR 曲线揭示了均匀量化器的根本缺陷：它是一个"只为满幅设计"的系统，信号一旦衰减，SNR 就以极快的速度崩塌。 μ 律量化器的曲线则近似水平，本质上是把动态范围的压力从量化位数转移到了压缩参数 μ 上。实验中用数字验证了一个在工程上意义重大的结论：6-bit μ 律在大动态范围内的表现，需要 11~12 bit 的均匀量化器才能达到相当水平。

Code

```
% Problem 2: SNR Comparison - Uniform Quantization vs mu-law Quantization
% Uses: mulaw.m, mulawinv.m, fxquant.m, SNR.m

[audio_data, ~] = audioread('s5.wav');
x_original = audio_data(1300:18800);

%% Part (a): 验证 6 dB/bit 规律 (8-bit vs 9-bit 均匀量化)
xh_8 = fxquant(x_original, 8, 'round', 'sat');
xh_9 = fxquant(x_original, 9, 'round', 'sat');

snr_8 = SNR(xh_8, x_original);
snr_9 = SNR(xh_9, x_original);

fprintf('=== Part (a): 6 dB/bit 规律验证 ===\n');
fprintf('8-bit 均匀量化 SNR : %.2f dB\n', snr_8);
fprintf('9-bit 均匀量化 SNR : %.2f dB\n', snr_9);
fprintf('SNR 增量           : %.2f dB (理论值  $\approx$  6 dB) \n\n',
snr_9 - snr_8);
```

```

%% Part (b): SNR vs  $1/\sigma_x$  半对数曲线 (动态范围 64:1)
% 衰减因子:  $2^0 \rightarrow 2^{-12}$ , 共 13 个电平
factors = 2.^(0:-1:-12);

bits_uniform = [10, 9, 8, 7, 6];
mu_values     = [100, 255, 500];
B_mu          = 6;

snr_uniform = zeros(length(bits_uniform), length(factors));
snr_mulaw   = zeros(length(mu_values),    length(factors));
inv_sigma_x = zeros(1, length(factors));

for i = 1:length(factors)
    x_scaled      = x_original * factors(i);
    inv_sigma_x(i) = 1 / rms(x_scaled);

    for b = 1:length(bits_uniform)
        xh = fxquant(x_scaled, bits_uniform(b), 'round', 'sat');
        snr_uniform(b, i) = SNR(xh, x_scaled);
    end

    for m = 1:length(mu_values)
        mu      = mu_values(m);
        y_comp  = mulaw(x_scaled, mu);
        y_quant = fxquant(y_comp, B_mu, 'round', 'sat');
        xh_mu   = mulawinv(y_quant, mu);
        snr_mulaw(m, i) = SNR(xh_mu, x_scaled);
    end
end

% --- 绘图 ---
figure('Name', 'SNR 动态范围对比');
hold on;

clr_uni = [0.09 0.47 0.69;
           0.20 0.63 0.17;
           0.55 0.34 0.71];

```

```

        0.89 0.47 0.06;
        0.68 0.18 0.18];

for b = 1:length(bits_uniform)
    semilogx(inv_sigma_x, snr_uniform(b, :), '-^', ...
        'Color', clr_uni(b, :), 'LineWidth', 1.6, 'MarkerSi
ze', 5, ...
        'DisplayName', sprintf('%d-bit 均匀量化', bits_unifo
rm(b)));
end

%  $\mu$ 律量化
clr_mu = [0.00 0.75 0.75;    %  $\mu=100$ 
          0.95 0.60 0.00;    %  $\mu=255$ 
          0.80 0.20 0.60];   %  $\mu=500$ 

mu_labels = {' $\mu=100$ ', ' $\mu=255$ ', ' $\mu=500$ '};
for m = 1:length(mu_values)
    semilogx(inv_sigma_x, snr_mulaw(m, :), '-.d', ...
        'Color', clr_mu(m, :), 'LineWidth', 2.0, 'MarkerSiz
e', 6, ...
        'DisplayName', sprintf('6-bit  $\mu$ 律 (%s)', mu_labels
{m})));
end

hold off;
grid on; box on;
title('均匀量化与  $\mu$  律量化 SNR 随动态范围的变化');
xlabel('1/\sigma_x (对数刻度)');
ylabel('信噪比 SNR (dB)');
legend('Location', 'southwest', 'NumColumns', 2);

```

Problem 3

自适应量化需要实时估计信号的局部标准差 $\sigma[n]$ 以进行增益均衡。本实验分别采用 IIR 滤波器（时间常数 $\alpha = 0.9, 0.99$ ）和 FIR 矩形窗（窗长 $M = 10, 100$ ）来计算方差，绘制原始语音与估计出的标准差包络线，并展示增益均衡后的信号 $x[n]/\sigma[n]$ 。

IIR 滤波器（指数衰减窗）——差分方程：

$$\sigma^2[n] = \alpha \cdot \sigma^2[n-1] + (1-\alpha) \cdot x^2[n-1]$$

α 接近 1 时，历史数据权重大，估计平滑但响应慢； α 接近 0 时，响应快但估计噪声大。

FIR 滤波器（矩形窗）——滑动平均：

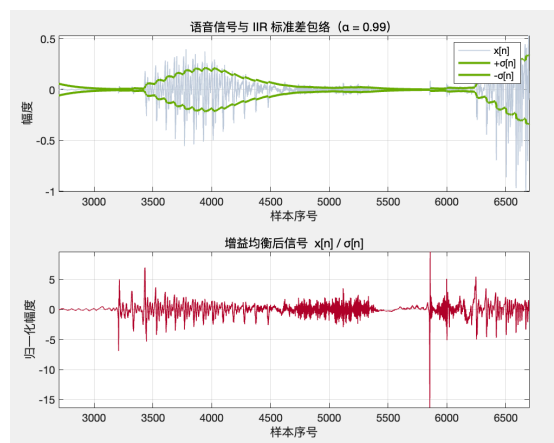
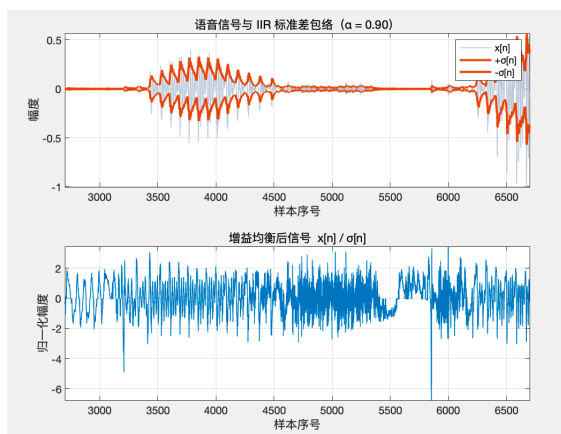
$$\sigma^2[n] = \frac{1}{M} \sum_{k=0}^{M-1} x^2[n-k]$$

等权重窗口，M 越大估计越平滑，但延迟也越大。

增益均衡：

$$x_{\text{eq}}[n] = \frac{x[n]}{\sigma[n] + \varepsilon}$$

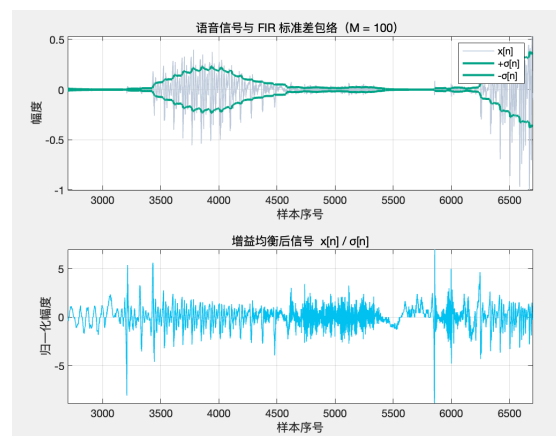
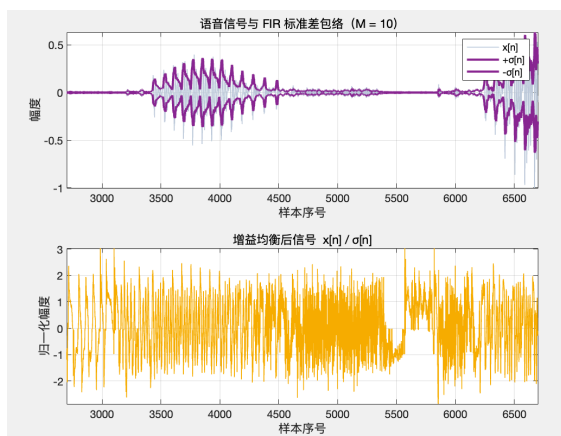
其中 $\varepsilon = 10^{-6}$ 防止静音段除零。



(a) IIR 滤波器对比 ($\alpha = 0.9$ vs $\alpha = 0.99$):

特性	$\alpha = 0.9$	$\alpha = 0.99$
记忆长度	短 (约 10 帧)	长 (约 100 帧)
包络追踪速度	极快，紧贴波形	慢，高度平滑
均衡后信号	幅度被"压平"，静音段底噪被过度放大	保留语音自然起伏，听感更好
实用性评价	均衡彻底但引入伪噪声	均衡与自然度折中，更适合实用

$\alpha = 0.9$ 的包络线对语音幅度变化的反应极快，但在静音段会过度放大背景噪声，导致静音段听起来"嘈杂"。 $\alpha = 0.99$ 包络极其平滑，适应速度较慢，在均衡效果与声音自然度之间取得了较好的折中。



(b) FIR 矩形窗与 IIR 滤波器的对比：

- $M = 10$ 的 FIR 表现类似 $\alpha = 0.9$ 的 IIR：响应快，但均衡过激。
- $M = 100$ 的 FIR 表现类似 $\alpha = 0.99$ 的 IIR：包络平滑，均衡温和。

核心差异——包络平滑性： 由于 FIR 使用矩形窗（等权重），当语音中出现能量极大的突变样本（如爆破音）进入或离开滑动窗口时，FIR 估计出的标准差包络上会出现生硬的"阶梯状折角"。相反，IIR 滤波器由于使用"指数衰减窗"，历史数据的影响力平滑淡出，因此其包络线过渡极其圆滑连续，更不易引入额外的调制失真。

在自然语音处理场景下，**IIR ($\alpha = 0.99$) 是更优的选择。**

小结

自适应量化的难点不在于算法本身，而在于"记忆长度"的权衡——滤波器追踪得越快，均衡越彻底，但静音段的底噪也会被同等程度地放大，听感反而变差。IIR 与 FIR 的本质差异体现在包络的过渡方式上：矩形窗的等权重特性使包络在突变样本进出窗口时产生硬折角，而指数窗的渐进衰减让包络始终保持平滑，这一差异在实际语音编解码器设计中直接决定了听觉质量。

Code

```
% Problem 3: Adaptive Quantization via Local Variance Estimation
% Uses: s5.wav

[audio_data, ~] = audioread('s5.wav');

% 去除直流分量
```

```

x = audio_data - mean(audio_data);

% 绘图区间：样本 2700 ~ 6699
idx    = 2700:6699;
x_plot = x(idx);

% 静音段防除零偏置
epsilon = 1e-6;
clr_bg = [0.75 0.80 0.85];

%% Part (a): IIR 滤波器（指数衰减窗）估计局部方差
% 差分方程：sigma^2[n] = alpha*sigma^2[n-1] + (1-alpha)*x^2
[n-1]

alphas    = [0.9, 0.99];
clr_iir   = [0.85 0.33 0.10;    % alpha=0.9  包络
             0.47 0.67 0.19]; % alpha=0.99 包络
clr_eq_iir = [0.00 0.45 0.74;    % alpha=0.9  均衡信号
             0.64 0.08 0.18]; % alpha=0.99 均衡信号

for i = 1:length(alphas)
    alpha = alphas(i);

    var_iir = zeros(size(x));
    for n = 2:length(x)
        var_iir(n) = alpha * var_iir(n-1) + (1 - alpha) * x
(n-1)^2;
    end
    sigma_iir = sqrt(var_iir);
    sigma_plot = sigma_iir(idx);
    x_eq       = x_plot ./ (sigma_plot + epsilon);

    figure('Name', sprintf('IIR  α=%.2f', alpha));

    % 语音波形 + 标准差包络
    subplot(2, 1, 1);
    plot(idx, x_plot, 'Color', clr_bg, 'LineWidth', 0.8);
    hold on;

```



```

    plot(idx, sigma_plot, 'Color', clr_iir(i,:), 'LineWidth', 2.0);
    plot(idx, -sigma_plot, 'Color', clr_iir(i,:), 'LineWidth', 2.0);
    hold off;
    title(sprintf('语音信号与 IIR 标准差包络 ( $\alpha = %.2f$ ) ', alpha));
    ylabel('幅度'); xlabel('样本序号');
    legend('x[n]', '+ $\sigma[n]$ ', '- $\sigma[n]$ ', 'Location', 'northeast');
    grid on; axis tight;

    % 增益均衡后信号
    subplot(2, 1, 2);
    plot(idx, x_eq, 'Color', clr_eq_iir(i,:), 'LineWidth', 0.8);
    title('增益均衡后信号 x[n] /  $\sigma[n]$ ');
    ylabel('归一化幅度'); xlabel('样本序号');
    grid on; axis tight;
end

%% Part (b): FIR 矩形窗估计局部方差
%  $\sigma^2[n] = (1/M) * \sum_{k=0}^{M-1} x^2[n-k]$ 

M_values = [10, 100];
clr_fir = [0.49 0.18 0.56; % M=10
           0.19 0.63 0.53]; % M=100
clr_eq_fir = [0.93 0.69 0.13; % M=10
              0.30 0.75 0.93]; % M=100

for i = 1:length(M_values)
    M = M_values(i);

    % filter 实现因果滑动平均
    var_fir = filter(ones(1, M) / M, 1, x.^2);
    sigma_fir = sqrt(var_fir);
    sigma_plot = sigma_fir(idx);
    x_eq = x_plot ./ (sigma_plot + epsilon);

```

```

figure('Name', sprintf('FIR M=%d', M));

% 语音波形 + 标准差包络
subplot(2, 1, 1);
plot(idx, x_plot, 'Color', clr_bg, 'LineWidth', 0.8);
hold on;
plot(idx, sigma_plot, 'Color', clr_fir(i,:), 'LineWidt
h', 2.0);
plot(idx, -sigma_plot, 'Color', clr_fir(i,:), 'LineWidt
h', 2.0);
hold off;
title(sprintf('语音信号与 FIR 标准差包络 (M = %d) ', M));
ylabel('幅度'); xlabel('样本序号');
legend('x[n]', '+σ[n]', '-σ[n]', 'Location', 'northeas
t');
grid on; axis tight;

% 增益均衡后信号
subplot(2, 1, 2);
plot(idx, x_eq, 'Color', clr_eq_fir(i,:), 'LineWidth',
0.8);
title('增益均衡后信号 x[n] / σ[n]');
ylabel('归一化幅度'); xlabel('样本序号');
grid on; axis tight;
end

```